01 · 회의 개요

회의의 목적과 결론

3명의 사전 준비 방향을 맞추고, 기술 검증과 GitHub 협업 실습을 통해 해커톤 당일의 통합 위험을 줄이는 회의였습니다.

항목	내용
회의	2026년 10월 28일(수) · 녹취 약 2시간 33분 40초
참석	손희영님(행정 팀장·UX/UI), 정민규님(Product·기획), 김동조님(개발)
기준	삼성 녹음 AI 전사본(292쪽), 회의 후 당사자 확인, 해커톤 목표 타임테이블 이미지
기록 방식	발언을 주제별로 재구성. 확정·조건부 합의·제안·미정 구분, 녹취 인식 오류는 문맥 범위에서만 교정

이번 회의의 핵심 결과

종합

- **코드 기반** — KONNECT 전체 재사용은 지양. 김동조님이 **10/29(목) 정오까지** 최소 실행·배포 테스트 후 재사용 판단.
- **GitHub** — 각자 작업 브랜치와 PR 중심으로 진행하고 **손희영님이 최종 Merge** 담당.
- **기획·디자인** — MVP PRD와 화면 명세를 먼저 명확히 하고, Figma는 7개 페이지를 4개로 간소화하는 방향 동의.
- **AI 협업** — 서로 다른 AI 코딩 도구의 변경 충돌 방지를 위해 `TEAM_WORKFLOW.md` 준비.
- **일정** — 첫날 핵심 기능이 실제로 작동하도록 노력하되 첨부 시간표는 **이상적 목표안**이며 확정 근무표가 아님.

지금도 확정되지 않은 것

별도 결정 필요

Vite 도입, `develop` 브랜치 사용, PR 공식 Approve 필수 여부, DB·RAG·GPU 사용, 최종 배포 책임자 및 제출 담당자, 본 프로젝트의 세부 기술 스택.

참고: 아래 시간은 녹음 시작을 00:00으로 본 상대 시각입니다. 일부 자동 전사 문장이 부정확하므로 세부 명령어·서비스명을 원문 인용처럼 확정하지 않았습니다.

2시간 33분 회의의 전체 지도

원본 전체를 9개 구간으로 나눈 목차입니다. 특히 GitHub 실습이 회의 후반 약 1시간 이상을 차지했습니다.

시간(녹음 기준)	논의 흐름	핵심 쟁점
00:00~00:14	일정·Figma·Discord	목표 타임테이블, 화면 명세 중심 프로세스, 7→4페이지 개편, 채널 수 과다
00:14~00:30	재사용·스택·GPU	KONNECT 최소 골격 재사용, FastAPI·배포 후보, Vite 논의, GPU의 추론/서버 활용
00:30~00:40	협업 원칙·공유 도구	서로 다른 AI 도구 사용, 공통 규칙 필요, GitHub·공유자료 정리
00:40~01:00	AI·RAG·기능 깊이·기획	모델 전환, 데이터 관리, 개인화 결과 검증, Deep Interview/Research
01:00~01:13	멘토링·3일 작업 계획	금요일 PRD 완료 목표, 멘토링 활용, 야간 AI 작업과 휴식
01:13~01:40	GitHub 실습 시작	작업 브랜치, Commit·Push, PR 생성, 코딩 AI와 터미널 접근
01:40~02:13	PR·Merge 실습	동일 파일과 신규 파일 변경, 검토·병합 차이, 충돌·최종 반영 확인
02:13~02:30	브랜치 보호·권한 점검	`main` 직접 반영 문제 확인, 저장소 Public 전환, 팀장 병합 제한 확인
02:30~02:33	후속 정리·gstack	공통 지침 작성, GPU 검토, gstack 활용 자료 공유 예정

읽는 방법

회의록 범위

이후 페이지는 시간순 발언록이 아니라 **주제별 정리**입니다. 실제 합의와 제안을 혼동하지 않도록 각 항목에 결정 수준을 표시하고, 필요한 경우 시간 구간을 병기했습니다.

기록에서 제외한 내용

의도적으로 제외

입장·이동·컴퓨터 연결 과정의 일상적 대화, 개인적 잡담, 중복 설명, 팀 의사결정과 직접 무관한 사담은 요지에 영향을 주지 않아 생략했습니다.

왜 첫날 구현까지 목표로 했는가

2박 3일 중 첫날 기획만 끝내면 QA·통합·발표 시간이 급격히 줄어든다는 문제의식이 있었습니다.

첫날: PRD를 오후 3시경까지 만드는 목표

권장 시나리오

- 주제 발표·심사 조건 확인 → 아이디어 선정 → Deep Interview 활용 → **최소 기능·사용자 시나리오·성공 기준이 있는 MVP PRD** 제작.
- PRD가 나온 뒤 정민규님이 **화면 명세·User Flow**를 중심으로 정리하고, 디자인은 핵심 화면 위주로 구성.
- 처음부터 전체 화면을 정교하게 와이어프레이밍하면 시간이 부족하므로, 중요한 화면·복잡한 전환 위주로 작업을 줄이는 방향.
- 금요일 저녁 UI 초안·개발 환경을 맞추고 **밤까지 첫 핵심 기능의 실제 작동**을 확보하는 것이 목표.

토요일·일요일: 개발 → QA → 시연 안정화

시간표는 목표안

- 토요일 오전: 전날 AI 작업 결과 검토, 핵심 기능 구현과 디자인 고도화.
- 토요일 오후: 구현·기능별 테스트·통합 QA, 이후 발표 자료 및 시연 흐름 준비.
- 일요일 오전: 야간 AI 검증 결과 확인, 치명적 오류·시연 준비에 집중한 뒤 제출.

멘토링 운영에 관한 논의 (01:01~01:09)

논의 / 최종 예약 미확정

- 멘토링은 기획이 어느 정도 구체화된 뒤 받는 것이 유리할 수 있다는 의견.
- 금요일 초기에 아이디어 범위와 판단 기준을 확인하고, 개발이 막혔을 때 기술적인 도움을 받는 방안 논의.
- 후반에 큰 방향을 뒤집으면 일정이 위험해지므로 **핵심 수정은 가능한 금요일에** 판단하자는 의견.
- 멘토링 횟수와 시간은 안내에 따라 조율하는 부분이 있어 **사용 시간·예약은 확정하지 않음**.

근무·야간 AI 작업

계획 전제

금·토·일 순작업 약 25~25.5시간(금 약 10h, 토 12~12.5h, 일 약 3h)을 가정했습니다. 야간에는 코드 생성·리뷰·회귀 테스트 같은 독립 AI 작업을 활용할 수 있지만, 실제 사람의 야간 근무·귀가·수면 일정은 고정하지 않았습니다.

Figma 구조와 화면 명세 협업

디자인 자체의 완성도뿐 아니라 화면 명세와 개발 인계가 일치하도록 하는 것이 주요 목적이었습니다.

제안된 작업 순서 (00:02~00:07)

방향 동의

1. 주제·MVP 범위와 PRD 확정
2. 정민규님 중심의 화면 명세 및 User Flow 정리
3. 화면 명세를 기준으로 **핵심 화면 UI 초안** 제작
4. 명세 대비 구조·동작 검토, 필요한 수정 반영
5. 디자인 고도화와 개발을 병렬로 진행

Figma 페이지 단순화

구조 변경 동의

- 기존 KONNECT와 유사하게 준비된 **7개 페이지를 4개 페이지로 줄이는 방향**에 동의.
- 안내, MVP Flow(가칭), Screens 등 목적에 맞는 구조를 논의했으며 **정확한 네 개 페이지명은 이 회의록에서 확정 명칭으로 단정하지 않음**.
- Screens 아래 Working / Review / Ready for Dev와 같이 작업 상태를 구분하는 방식이 제안됨.
- 이 상태 구분은 디자인과 개발 인계 시 혼동을 줄이려는 취지. 실제 생성·적용 여부는 별도 확인 필요.

디자인 품질·합의 범위

방향 / 일부 제안

- 손희영님은 화면 명세가 정리되기 전이라도 **스타일·방향성을 어느 정도 일찍 결정**해야 재작업이 적다고 언급.
- 과거 Figma에서 폰트·글자 맞춤 문제를 경험한 만큼 타이포그래피 선택과 검토 필요성을 논의.
- Inter 같은 기본 폰트 외에 Google Fonts 활용을 제안했으나 **특정 폰트는 확정하지 않음**.
- 와이어프레임 전체를 세밀하게 만들기보다 핵심 화면과 상호작용을 우선하는 방법을 검토.

“완성”의 기준을 어떻게 볼 것인가

회의에서는 단순한 기능 연결보다, 실제 사용자 조건에 맞게 동작하고 이를 테스트할 수 있는지가 중요하다는 이야기가 반복됐습니다.

“동작하는 데모”와 “의도대로 동작하는 기능”은 다름

기능 품질 관점

- 프론트엔드와 API를 단순 연결해 페이지를 띄우는 데 비해, **사용자 조건에 맞는 올바른 결과를 반환**하도록 만드는 작업이 더 어렵다는 점을 논의.
- 예: 같은 장소·시간 조건에서도 사용자가 역사에 관심 있다고 말한 경우와 그렇지 않은 경우 **추천 결과가 달라져야 한다**는 사례.
- 기능을 여러 개 만드는 대신 중요한 기능의 정확도·일관성·개인화 수준을 확보하는 방향에 공감.
- AI에 요청하는 프롬프트가 구체적이어야 하며, 개발 후 실제 입력·출력 비교 검증이 필요하다는 취지.

API · 데이터 처리에서 제기된 문제

대안 검토

- 외부 API의 실제 데이터 품질, 호출 가능 범위, 처리 속도와 안정성은 새 주제가 정해진 뒤 확인해야 함.
- 단순한 로컬 JSON, SQLite, Supabase, 캐시 등의 접근이 언급됐지만 실제 데이터 설계·선택은 미정.
- 데이터를 빠르게 저장·조회하는 것과 LLM 결과의 적합성·품질을 검증하는 것은 다른 문제로 논의됨.
- 로그인·장기 기억·복잡한 파이프라인은 KONNECT 당시 과제 요구였으므로, 이번 해커톤에서는 필수 기능으로 가정하지 않음.

Deep Interview / Deep Research 활용 (00:57~01:00)

활용 제안

- Deep Interview로 목표·대상·범위·제외 범위·성공 기준을 질문·응답 방식으로 구체화하는 방안.
- 그다음 Deep Research 등으로 필요한 사실이나 시장·기술 가정을 검증하는 흐름 언급.
- 도구의 정확한 조합과 사용 시간은 금요일 주제에 따라 선택하기로 하며, 자동으로 모든 도구를 적용하기로 확정한 것은 아님.

KONNECT를 어디까지 재사용할 것인가

선호가 다른 지점을 논쟁으로 끝내지 않고, 목요일 정오까지 작은 실행·배포 검증을 해 본 뒤 결정하기로 했습니다.

김동조님의 출발 제안 (00:14~00:22)

제안

- KONNECT 코드를 분석했더니 파일 수와 내부 로직이 많아 **그대로 들고 오기에는 복잡하다**고 판단.
- 백엔드(FastAPI) 등 쓸 수 있는 골격은 최대한 유지하고, 프론트엔드는 필수 구조만 남기는 최소 베이스를 제안.
- 원래 프로젝트를 참고 가능한 별도 브랜치나 자료로 보존하고, 새 주제에서 필요한 기능만 선택적으로 활용하자는 취지.
- AI 호출 체인·복잡한 기능을 줄이고 새 기능은 작은 단위로 추가하는 방식 제시.

정민규님의 우려와 팀의 조정

이견을 시험으로 해결

- 다른 프로젝트 코드를 가져다 쓰면 **불필요한 코드 의존성과 수정·통합 비용**이 생길 수 있다는 우려.
- 새 해커톤 주제가 KONNECT와 완전히 다르면, 기존 코드보다 최소 구조를 처음부터 만드는 것이 더 빠를 수도 있음.
- 손희영님도 동조님의 최소 구조 테스트를 통해 실제 쓸 수 있는지 확인한 뒤 진행하자는 방향에 동의.

검증 기준과 기한 (00:20~00:23)

조건부 합의

- **김동조님: 10/29(목) 12:00까지** 최소 실행 및 배포 동작 테스트.
- 기능 전체를 만들기보다 프론트·백엔드 연결과 배포 후보에서 기본 화면이 뜨는지 확인.
- 성공하면 필요한 부분 재사용을 검토하고, 문제가 크면 새로 구성하는 선택지를 열어 둠.
- 회의 당시에는 **테스트 계획과 기한이 확정**된 것이지, 실제 성공이나 구조 안정성이 검증된 것은 아님.

잠정 기술 후보

잠정

프론트: HTML/CSS/Vanilla JS(+Vite 미정) · **백엔드:** FastAPI 유력 · **배포:** Vercel/Render 유력 · **DB:** Supabase는 필요할 때 검토. 어떤 조합도 주제 발표 전 최종 확정하지 않았습니다.

AI API · RAG · GPU 논의의 실제 결론

"쓸 수 있다"와 "해커톤에서 쓰기로 확정했다"는 다릅니다. 기술 가능성을 논의했지만 대부분 주제·테스트에 따라 달라집니다.

네이토 API와 모델 전환 (00:16~00:18, 00:39~00:50)

기본 후보 / 검증 전

- 네이토 API가 서비스용 LLM 호출의 기본 후보로 언급됨. 기존 OpenAI API 또는 다른 모델과 선택·전환하는 구성도 제안됨.
- 어느 한 모델이 장애를 일으킬 경우 다른 경로로 전환할 수 있도록 하는 취지. **이중화 구현 자체는 아직 테스트되지 않음**.
- 제공 토큰량이 충분하지 않을 수 있다는 우려가 있었으나, 실제 지급 단위·계정 배분·소진 정책은 안내 재확인 필요.

RAG: 의견 차이를 명확히 기록

의견 차이 / 미정

- **손희영님:** 서비스의 답변·추천 신뢰도를 위해 RAG가 중요하다는 의견을 강하게 제시.
- **정민규님:** 서비스 주제와 실제 데이터 요구에 따라 필요성이 달라질 수 있다는 입장.
- **김동조님:** RAG는 데이터가 마련돼야 실효성이 있으므로 주제·데이터 형태를 보고 붙일 수 있다는 취지.
- **결론:** 금요일 주제가 정해지기 전까지 RAG 필수 도입은 확정하지 않음.

GPU V100 활용: 서로 다른 두 가지 용도 (00:16~00:28, 00:46~00:47)

추가 확인 필요

- ① **로컬 LLM 추론:** 외부 토큰 비용을 줄이는 가능성을 논의했으나 설정 부담·속도·품질 우려가 제기됨.
- ② **배포 서버:** 무료 Vercel/Render 등에서 발생할 수 있는 초기 응답 지연을 피하는 용도로 사용할 수 있는지 질문.
- 그러나 GPU 제공과 상시 웹서버 운영 가능 여부는 별개이므로, 실제 접근 권한·네트워크·운영 정책 확인이 선행돼야 함.
- **손희영님·정민규님**이 추가 검토하며 도입 자체는 미정.

팀 공통 규칙·AI·Discord

기술 스택을 확정하는 것만큼 “누가 어떤 변경을 어디에 남기는가”를 명확히 하는 것이 협업 위험을 줄이는 핵심이었습니다.

AI 코딩 도구별 작업 충돌 문제 (00:30~00:33)

공감대

- 팀원들이 Codex, Claude Code, Gemini(Antigravity) 등 **서로 다른 AI 코딩 도구**를 활용할 수 있으므로 동일 파일의 예상치 못한 수정·리팩터링을 경계.
- 코딩 AI가 변경 범위 밖 파일이나 공통 설정을 건드리면 병합 후 문제가 생길 수 있다는 위험을 공유.
- 토큰 소비를 줄이고 작업 인계를 명확히 하기 위해 프로젝트 구조·규칙·수정 범위를 문서화할 필요성 공감.

TEAM_WORKFLOW.md 작성 방향

작성 합의 / 세부 제안

- 기술 스택, 저장소 구조, 작업 분담, AI 도구의 코드 수정 원칙, PR·Merge 절차, API 규칙, 배포 책임을 짧게 정리.
- AI 도구가 읽을 수 있는 프로젝트 안내 파일(예: `AGENTS.md`, `CLAUDE.md`, `GEMINI.md`)도 언급됐으나, 구체적인 파일 구성·자동 적용 방식은 확정하지 않음.
- 실제 해커톤에서는 문서의 확정된 규칙과 미정 결정을 구분해 유지하는 것이 중요.

Discord·공유 문서 운영 (00:09~00:12, 00:34~00:37)

문제 제기 / 미정

- Discord 채널이 너무 많아서 정보를 찾기 어렵다는 문제를 제기했고, 기획·디자인·개발 영역의 채널 단순화를 논의.
- 공지·자료·제출물 등 용도에 따라 보존할 채널과 합칠 채널을 구별할 필요.
- 별도 공유 자료 접근 권한과 GitHub 초대 확인도 진행. **최종 채널 개편 목록은 회의에서 확정되지 않음**.

gstack (02:31~02:32)

후속 검토

정민규님이 gstack 관련 GitHub/WikiDocs 자료와 해커톤에 필요한 일부 활용법을 검토해 **다음 날 자료를 공유**하겠다고 언급했습니다. 무조건 전 기능 적용하자는 결정은 아닙니다.

작업 브랜치와 PR·Merge의 의미 확인

회의 후반의 핵심은 명령어 외우기가 아니라, 서로 만든 변경사항이 언제 어떻게 공통 코드에 들어가는지 직접 확인하는 실습이었습니다.

실습 시작과 단계 (01:13~01:56)

실습 사실

1. GitHub 실습 저장소 `git-practice` 접근·클론, 각자 작업 환경 준비.
2. 개별 브랜치 생성 후 간단한 HTML/문서 수정 → Commit → Push를 실습.
3. 작업 브랜치 변경사항을 공통 브랜치로 가져오기 위한 **Pull Request(PR)** 생성.
4. PR 생성 후 실제 Merge 전후 상태를 비교하고, 승인·병합·로컬 최신화가 별개 단계인지 확인.
5. 동일 파일의 내용 변경과 **서로 다른 파일·기능 단위의 반영**을 구분해 실습.

실습 중 드러난 혼동

학습·문제 발견

- GitHub 웹 화면에서 파일을 직접 편집하는 것과 **브랜치에서 작업하고 PR로 병합하는 것**의 차이를 여러 차례 확인.
- PR이 열려 있다는 것과 `main`에 실제 코드가 반영됐다는 것을 구별할 필요를 확인.
- 같은 이름의 파일을 여러 사람이 변경할 때 병합 결과를 어떻게 검토해야 하는지 논의.
- 기능별로 분리된 파일/브랜치도 최종 통합이 필요하다는 점을 시나리오로 점검.

GitHub Issue 사용에 대한 논의 (02:15)

권장안

“Issue 작성 → 작업 브랜치 → 코드 변경 → PR → 최종 Merge”라는 방법이 제안됐으나, **Issue를 매 작업마다 반드시 만들어야 한다고 확정하지는 않았습니다.** 이슈 기록이 변경 이력을 남기는 데 도움이 된다는 데 의견이 있었습니다.

브랜치 보호와 최종 병합 책임

이번 실습에서 가장 중요한 발견은 “PR을 쓰자는 규칙”과 “정말 다른 사람이 main에 병합할 수 없는 설정”이 다른 문제라는 점이었습니다.

권한 문제 발견 (02:03~02:22)

문제 발견

- 정민규님은 “팀장이 확인하기 전에는 일반 팀원이 `main`에 변경을 합칠 수 없어야 한다”는 동작을 반복 확인.
- 비공개 실습 저장소에서는 예상과 달리 직접 반영 또는 병합이 가능해 보이는 상황이 발생해 설정 문제를 점검.
- 손희영님·김동조님과 브랜치 보호와 권한의 차이를 논의했고, 저장소 공개 전환과 보호 규칙 재설정을 검토.

실습 후반의 결과 (02:22~02:29)

실습 결과 / 운영 확정

- 실습 저장소를 **Public**으로 바꾸고 팀장 중심의 `main` 병합 제한을 설정한 것으로 회의 후 정민규님이 확인함.
- PR을 통한 병합 흐름과 팀장 최종 Merge 담당 역할은 합의.
- 다만 각 계정에 실제로 적용된 제한·규칙 종류 및 해커톤 본 저장소에도 동일하게 적용되는지는 별도 확인 대상으로 남김.

명확히 확정된 것과 그렇지 않은 것

결정 구분

구분	현재 판단
확정	손희영님이 공통 코드의 최종 Merge 담당
큰 흐름 합의	개인 브랜치 작업 → Push → PR → 변경사항 확인 → 손희영님 최종 Merge
아직 미정	PR 공식 Approve를 필수 조건으로 할지, `develop` 브랜치를 사용할지
주의	Public 저장소와 권한 보호는 별도 문제. 실제 토큰·API 키 등은 저장소에 올리지 않기

실습 기록은 자동 전사에 따른 세부 버튼·명령어 오류 가능성이 있어, 명령 실행을 전부 성공한 것으로 단정하지 않았습니다.

10월 29~30일에 이어서 해야 할 일

사전회의의 성과는 문서 자체가 아니라 목요일의 실행 테스트와 금요일의 빠른 의사결정으로 이어질 때 완성됩니다.

담당자별 후속 작업

실행 준비

담당	할 일	기한·상태
김동조님	KONNECT 최소 골격 실행·배포 시험, 새 프로젝트 재사용 범위·문제점 공유	10/29(목) 12:00까지 · 합의
손희영님	V100 서버 제공 정보·활용 방식 추가 확인, Figma 4페이지 방향 적용 검토	추가 확인 · 적용 상태 미확인
정민규님	회의록·공통 개발 지침 초안 정리, gstack 활용 자료 추가 공유	회의 후 / 10/29 검토
팀 전원	10/29 테스트 결과 확인, 금요일 주제 발표 후 기술 스택·기능 범위 결정	10/30 주제 발표 후
팀 전원	최종 배포·발표자료 취합·제출 담당자 지정	Discord 또는 금요일 현장

팀에 질문할 항목: 우선순위별

정리 필요

- **즉시:** 목요일 정오 실행·배포 테스트가 성공했는지, 재사용·새 구축 중 어느 방향인지.
- **GitHub:** `main + feature`만 쓸지, `develop`을 더 둘지. Approve가 필수인지. 본 저장소 보호 설정은 어떻게 할지.
- **기술:** Vanilla JS + Vite 여부, FastAPI/Render/Vercel 채택, Supabase · 로그인·RAG 실제 필요성.
- **운영:** 네이토 토큰·사용량 관리, GPU 사용 여부, 무료 배포 지연 대응, 최종 시연/제출 책임.

기억할 기준

문서 인계

회의에서 나온 의견을 전부 실행 규칙으로 만들지 않습니다. **확정 사항은 TEAM_WORKFLOW.md에**, 미정 사항은 질문 목록과 금요일 결정 표에 남깁니다. 기술 제안은 새 주제가 결정된 뒤 검증 가능한 최소 작업으로 전환합니다.

해커톤 전체 타임테이블

10/30(금)~11/1(일) · 순작업 약 25~25.5시간 가정.



이 이미지는 정민규님이 제안한 **이상적인 목표 일정**이며, 팀 전체가 고정 근무·작업 시간으로 승인한 확정표는 아닙니다.